React Interview Questions and Answers

Last Updated : 19 Sep, 2025

ReactJS is a component-based JavaScript library used to build dynamic and interactive user interfaces. It simplifies the creation of single-page applications (SPAs) with a focus on performance and maintainability. It is widely used by companies like Facebook, Instagram, Netflix, and Airbnb because of its flexibility and performance.

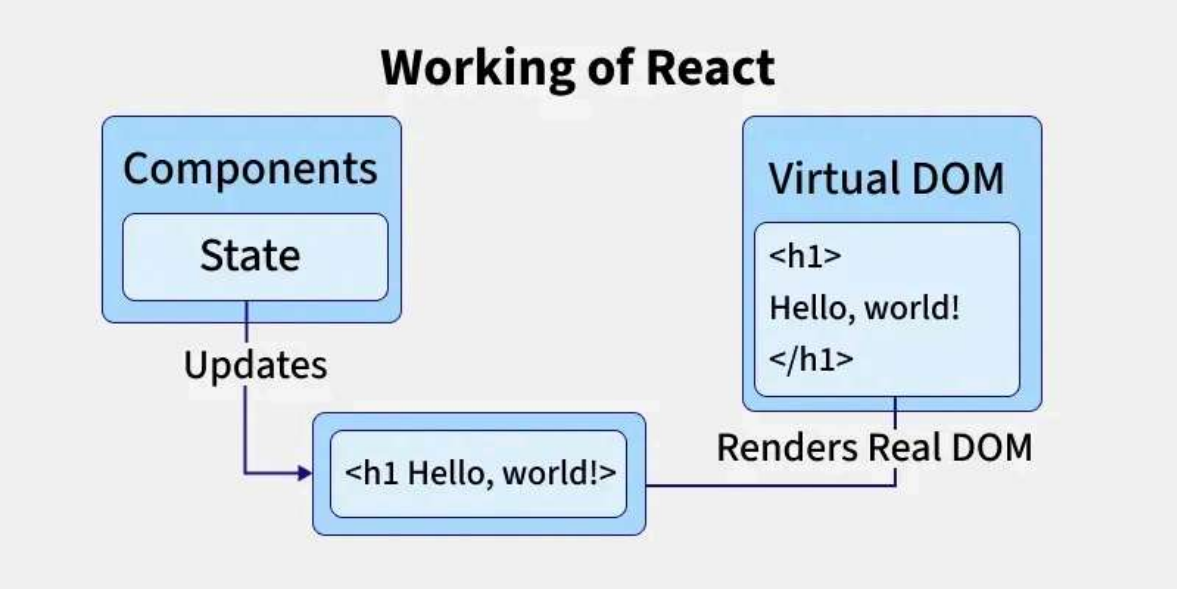
Key aspects of React are:

- **Reusable Components:** Speeds up development by allowing code reusability.
- **Declarative UI:** Makes it easier to describe how the UI should look based on the current state.
- **Cross-Platform Support:** Powers both web apps (React.js) and mobile apps (React Native).

Basic React Interview Questions

1. How does React.js work?

React.js works on a component-based architecture and uses a virtual DOM to efficiently update and render user interfaces.



- **Components**: UI is broken into reusable, independent pieces.
- **JSX**: Allows writing HTML-like code inside JavaScript for easier UI development..
- **Virtual DOM**: A lightweight copy of the real DOM that tracks changes.
- **Reconciliation**: Compares old and new Virtual DOM and updates only the changed parts in the real DOM.
- **One-way Data Flow**: Ensures predictable UI updates by passing data from parent to child via props.
- **State Management**: React automatically re-renders components when their state changes.

2. What is JSX and how is it converted into JavaScript?

- **JSX:** Syntax extension for JavaScript, mainly used with React.
- **HTML in JS:** Allows writing HTML-like code inside JavaScript for easier readability and maintenance.
- **Expressions:** Embed JavaScript expressions in JSX using {}.

Example of JSX: The name written in curly braces { } signifies JSX

```
const name = "Learner";
const element = (
    <h1>
        Hello,
        {name}.Welcome to GeeksforGeeks.
    </h1>
);
```

Browsers can't understand JSX directly. Instead, tools like Babel transpile it into plain JavaScript using React.createElement().

```
const element = <h1>Hello, Geeks!</h1>;
```

Babel converts it into:

```
const element = React.createElement("h1", null, "Hello, World!");
```

3. What is a React component?

A Component is one of the core building blocks of React. In other words, we can say that every application you will develop in React will be made up of pieces called components. Components make the task of building UIs much easier.

In React, we mainly have two types of components:

- **Functional Components:** Functional components are simply JavaScript functions. Initially, they were limited in terms of features like state and lifecycle methods. However, with the introduction of Hooks, functional components can now use state, manage side effects, and access other features that were once exclusive to class components.
- **Class Components:** Class components are more complex than functional components. They are able to manage state, handle lifecycle methods, and can also interact with other components. Class components can pass data between each other via props, similar to functional components.

4. Difference between functional and class component in React?

Functional Components	Class Components
A functional component is just a plain JavaScript pure function that accepts props as an argument	A class component requires you to extend from React. Component and create a render function
No render method used	It must have the render() method returning JSX
Also known as Stateless components	Also known as Stateful components

Functional Components	Class Components
React lifecycle methods (for example, componentDidMount) cannot be used in functional components.	React lifecycle methods can be used inside class components (for example, componentDidMount).
Constructors are not used.	Constructor is used as it needs to store state.
Uses hooks like useState for managing state.	Uses this.state and this.setState for state management

5. What are props and default props in React?

React allows us to pass information to a Component using something called props (which stands for properties). Props are objects which can be used inside a component. We can access any props inside from the component's class to which the props is passed. The props can be accessed as shown below:

```
this.props.propName;
```

Default props are fallback values assigned to a component's props when the parent does not provide them. They help prevent undefined errors and make components more robust. Default props can be defined using:

- **Class components:** ComponentName.defaultProps = { propName: defaultValue }
- **Functional components:** assigning default values in function parameters.

6. What is state in React and how do you update it?

State is the internal, mutable data of a React component that controls its behavior and rendering. When state changes, React re-renders the component to update the UI.

State is updated using setState() in class components or the useState setter function in functional components. Updates are asynchronous and can be triggered by events, user interactions, or API responses. When new state depends on the previous state, the functional form (setState(prev => ...)) should be used.

7. Difference Between Props and State in React?

PROPS	STATE
The Data is passed from one component to another.	The Data is passed within the component only.
It is Immutable (cannot be modified).	It is Mutable (can be modified).
Props can be used with state and functional components.	The state can be used only with the state components/class component (Before 16.0).

PROPS	STATE
Props are read-only.	The state is both read and write.
Example: Passing a title or onClick handler to a button component.	Example: A counter value that increases when you click a button.

8. What are fragments in React?

In React, fragments allow you to group multiple elements without adding extra nodes to the DOM. Normally, returning multiple elements requires a wrapper like a , which can create unnecessary DOM elements. Fragments solve this by letting you return multiple elements without extra wrappers.

- Fragments avoid extra DOM nodes.
- Short syntax: <> </> (cannot use attributes).
- Full syntax: <React.Fragment> </React.Fragment> (can use key).
- Useful in lists, tables, and grouping multiple elements.
- Helps keep the DOM clean and lightweight.

9. What is the difference between controlled and uncontrolled components?

Controlled Components	Uncontrolled Components
The React state controls the form input value.	The DOM manages the input value.
Every change in input updates the state via onChange.	React uses ref to access the current value when needed.
Provides full control over input data.	Less code, simpler for basic forms.
Useful for validation, conditional rendering, or complex forms.	Less control over validation and state management.

10. How does the DOM manage the input value in uncontrolled components?

React does not manage the form input's state. Instead, the DOM itself keeps track of the input's value. You can access the value when needed using refs

- Input values are stored in the DOM, not React state.
- Use React.createRef() or useRef() to access the current value.
- Useful when you don't need to react to every change or for simple forms.

React Hooks Interview Questions

11. What are Hooks in React and why were they introduced?

Hooks are special functions in React that let you use state, lifecycle methods, and other React features in functional components, which were previously only available in class components.

Why introduced:

- Before hooks, state and lifecycle methods could only be used in class components, leading to more boilerplate code and complex patterns.
- Hooks allow simpler, cleaner code using functional components.
- They promote reusability and composition of logic between components.

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

12. How does the useState hook work?

The useState hook allows you to add state to functional components. It returns a state variable and a setter function to update that state. When the state changes, React re-renders the component with the updated value.

Example:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

In this example:

- count is the current state value.
- setCount is the function used to update state.
- 0 is the initial value when the component mounts. When setCount is called, React re-renders Counter with the new count value.

13. What is useEffect in React and what is the role of its dependency array?

useEffect is a React hook that allows functional components to perform side effects such as fetching data, setting up subscriptions, or directly manipulating the DOM. It runs after the component renders, ensuring the UI is updated first.

The dependency array controls when the effect runs:

- [] : runs once after the component mounts (like componentDidMount).
- [dep1, dep2] : runs only when a dependency changes.
- Omitted (no array) : runs after every render.

useEffect can also return a cleanup function that runs on unmount or before the next effect, helping prevent memory leaks.

Example :

```
useEffect(() => {
  console.log('Effect runs');
  return () => console.log('Cleanup runs');
}, [dependency]);
```

14. What is the difference between useEffect and useLayoutEffect?

Painting in React refers to the browser process of drawing updated DOM changes onto the screen after React completes rendering

useEffect	useLayoutEffect
Runs after render and painting	Runs before painting, after DOM updates
Non-blocking	Blocking
Used for data fetching, subscriptions, timers	Used for reading layout, measuring DOM, or sync DOM updates
Safer for most side effects	Can slow rendering if heavy logic is used
Runs after every render by default (or based on dependency array)	Use when DOM measurements are needed before painting

15. What is the useContext hook and when should you use it?

The useContext hook allows functional components to consume values from a React Context without passing props through every level of the component tree (avoiding “prop drilling”).

When to use it:

- Sharing global data like theme, user info, or authentication status.
- Avoiding prop drilling through many component levels.
- When multiple components need access to the same state or data.

16. What is the useReducer hook and when is it preferred over useState?

The useReducer is a React hook used to manage complex state logic in functional components. It works similarly to Redux: you define a reducer function that takes the current state and an action, and returns a new state.

Example:

```
const [state, dispatch] = useReducer(reducer, initialState);

// Example usage:
dispatch({ type: 'increment' });
```

In this example:

- state is the current state.
- dispatch is used to send actions to update the state.

Preferred over useState when:

- State logic is complex or has multiple sub-values.
- Next state depends on previous state.
- You want centralized state management in a component.

17. What is the useRef hook and what are its common use cases?

The useRef hook creates a mutable object that persists across renders without causing re-renders. It is commonly used to access DOM elements or store mutable values.

Example:

```
import { useRef } from 'react';

function App() {
  const inputRef = useRef();

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}>Focus</button>
    </div>
  );
}
```

In this example:

- inputRef stores a reference to the input DOM node.
- Calling inputRef.current.focus() focuses the input without re-rendering the component.

Common use cases:

- Accessing DOM nodes directly (focus, scroll, measure).
- Storing mutable values that don't trigger re-renders.
- Keeping previous state values for comparison.

18. Explain the difference between useMemo and useCallback?

useMemo	useCallback
Memoizes computed values	Memoizes functions
Returns the result of a function	Returns the function itself
Avoids expensive recalculations on re-render	Prevents re-creating functions on re-render
Improves performance by caching values	Improves performance by avoiding unnecessary renders
Key idea: caches value	Key idea: caches function
Syntax: useMemo(() => computeValue(a, b), [a, b])	Syntax: useCallback(() => handleClick(id), [id])

19. What are custom hooks in React and how do you create one?

Custom hooks are reusable functions that let you extract and share logic between functional components. They are JavaScript functions whose names start with `use` and can call other hooks like `useState` or `useEffect`.

Example:

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(data => {
        setData(data);
        setLoading(false);
      });
  }, [url]);

  return { data, loading };
}

// Usage in a component
function App() {
  const { data, loading } = useFetch('https://api.example.com/data');

  if (loading) return <p>Loading...</p>;
  return <div>{JSON.stringify(data)}</div>;
}
```

In this example, `useFetch` is a custom hook that fetches data from an API and manages loading state. The hook is reused in `App` to keep the component clean and separate the data-fetching logic.

20. What are the rules of hooks and why are they important?

Hooks in React must follow certain rules to work correctly. These rules ensure that React can track state and effects reliably and prevent unexpected bugs.

- Always call hooks at the top level of your component or custom hook.
- Only call hooks from React functional components or custom hooks.
- Custom hooks should always start with `use`, like `useFetch` or `useForm`.

Why they are important:

- Preserve state and effect order between renders.
- Ensure components behave predictably.
- Prevent bugs and unexpected behavior when using hooks like `useState` and `useEffect`.

React State Management Interview Questions

21. What is State Management in React and what is the difference between Local and Global State?

State management refers to how an application handles and shares data across components. It ensures that the UI is updated correctly whenever the underlying data (state) changes. In React, state can be managed locally (inside components) or globally (shared across multiple components using context, Redux, Zustand, etc.).

Local State	Global State
Data managed within a single component.	Data shared across multiple components.
Controlled using useState or useReducer inside the component.	Managed using Context API, Redux, Zustand, etc.
Used for UI-related data like form inputs, modals, or toggles.	Used for app-wide data like authentication, user info, or theme

22. What are some popular state management solutions in React/Next.js?

There are several approaches and libraries for managing state in React and Next.js, depending on the scale and complexity of your application:

- **React built-in hooks:**useState, useReducer, useContext – for local or small-scale state management.
- **Context API:** Ideal for small to medium global state needs.
- **Redux / Redux Toolkit:** Widely used for large-scale applications with complex state logic.
- **Zustand:** Lightweight state management library for simplicity and performance.
- **Recoil / Jotai / MobX:** Alternative libraries offering fine-grained state control.
- **Server-side state (Next.js):**useSWR, React Query, or Server Actions for fetching and caching server data efficiently.

23. When should you use Redux over Context API?

Choosing between Redux and Context API depends on the size and complexity of your application:

Use Redux when:

- The application is large and complex.
- You need predictable state management with debugging tools (e.g., Redux DevTools).
- There are frequent state updates and many shared states across components.

Use Context API when:

- The application is small or medium-sized.
- You only need to share simple state, such as theme, language, or user info.

24. What is the difference between Client State and Server State?

Client State	Server State
Managed locally in the app	Managed on backend/server
Controlled by useState, useReducer, Context	Controlled via API or DB
Short-lived (browser session)	Persistent across sessions/users
Examples: form inputs, UI toggles	Examples: user data, product list

25. Explain the Context API in React.

The Context API in React is used to share data globally across components without prop drilling. It allows you to wrap components with a provider and access the shared value anywhere in the tree using useContext.createContext() : Creates a context.

- Avoids prop drilling.
- Uses Provider to supply data.
- Components consume data with useContext or Consumer.
- Best for global values like theme, auth, language.
- Often combined with useState/useReducer for updates.

26. What is the role of useReducer in state management?

The useReducer hook in React plays an important role in state management when the logic for updating state is complex or involves multiple actions. Unlike useState, which is best for simple state updates, useReducer organizes state transitions using a reducer function, making the code more predictable and easier to maintain. It is especially useful in larger applications or when managing related pieces of state.

- Manages complex state logic more effectively than useState.
- Uses a reducer function that takes state and action, then returns a new state.
- Returns [state, dispatch], where dispatch triggers actions.
- Helps keep state updates predictable and organized.
- Often used in forms, lists, or apps needing Redux-like patterns.

27. How do you handle persistent state in React apps?

Persistent state ensures that certain data remains available even after page reloads or across sessions. Common approaches include:

- **localStorage or sessionStorage:** Save state that persists across page reloads.
- **Combine state hooks with effects:** Use useState or useReducer together with useEffect to sync state with storage.
- **IndexedDB:** For larger or more complex client-side storage needs.
- **Backend storage:** Store state on a server/database via APIs for server-side persistence.
- **State management libraries:** Use Redux, Zustand, or Context API with persistence middleware.
- **Cookies:** Suitable for small pieces of data, like authentication tokens.

28. What is the difference between derived state and computed state in React?

In React, sometimes you need values that depend on other state or props. These values can be handled as derived state or computed state, but they work differently:

- **Derived state:** State calculated from props or other state. Should be avoided if it can be computed during render.
- **Computed state:** Calculated on the fly during render or using hooks like useMemo. Ensures data stays consistent without storing redundant state.

29. How can you optimize state updates for performance in React?

Optimizing state updates ensures that components re-render only when necessary, improving performance in React applications:

- Keep state localized to the smallest possible component.
- Use useMemo and useCallback to avoid unnecessary re-renders.
- Batch state updates where possible.
- Avoid deep nested objects in state; normalize the data.
- Use state management libraries efficiently (e.g., Redux selectors, Zustand slices).

30. How do you handle asynchronous state updates in React?

In React, state updates are asynchronous, so you need to handle them carefully to ensure predictable results:

- State updates (via `setState` or the setter from `useState`) do not happen immediately.
- Use the functional form of `setState` when the new state depends on the previous state:

```
setCount(prevCount => prevCount + 1);
```

- For multiple sequential updates, consider `useReducer` for predictable state transitions.
- Use effects (`useEffect`) to react to changes in state asynchronously.

React Rendering & Performance Questions:

31. Can you explain what the Virtual DOM is and how React uses it?

The Virtual DOM (VDOM) is a lightweight, in-memory representation of the real DOM in the browser. Instead of updating the actual DOM directly, which can be slow and costly, React first updates the Virtual DOM when a component's state or props change. React then compares the new Virtual DOM with the previous one using a process called diffing, identifies the minimum set of changes needed, and updates only those parts of the real DOM. This approach makes UI updates faster and more efficient by reducing unnecessary DOM manipulations. Key points:

- Virtual DOM is a JavaScript object representing the real DOM.
- React updates the VDOM first, not the real DOM.
- Uses a diffing algorithm to detect changes efficiently.
- Only the necessary parts of the real DOM are updated.
- Improves performance and user experience in dynamic applications.

32. What do you understand by reconciliation in React? Why is it important?

Reconciliation in React is the process of comparing the new Virtual DOM with the previous one to determine what has changed. React then updates only the necessary parts of the real DOM, instead of re-rendering the entire UI. This approach improves performance, rendering speed, and efficiency, ensuring a smooth and responsive user experience. Key points:

- Reconciliation is React's process to update the DOM efficiently.
- Compares new Virtual DOM with previous Virtual DOM using diffing.
- Updates only the changed parts of the real DOM.
- Improves performance, rendering speed, and efficiency.
- Ensures a smooth and responsive UI.

33. How does `React.memo` help improve performance?

React.memo is a higher-order component that prevents unnecessary re-renders of functional components. It re-renders a component only if its props have changed, which helps improve performance in components that render frequently or are part of large lists.

Example: If you have a large list of items and only one item changes, React.memo ensures that the other list items do not re-render, saving processing time and improving UI performance.

34. If I ask you to optimize a slow React application, what techniques would you use?

To improve performance in a slow React application, you can use several optimization techniques:

- Use React.memo, useMemo, and useCallback to avoid unnecessary re-renders.
- Split code using React.lazy and Suspense for on-demand component loading.
- Optimize large lists with react-window or react-virtualized.
- Avoid anonymous functions in render; use useCallback instead.
- Keep state localized as much as possible instead of placing everything in global state.
- Use proper key props when rendering lists.
- For server data, use libraries like React Query or SWR with caching to reduce unnecessary refetching.

35. What do you mean by code splitting and lazy loading in React?

Code splitting and lazy loading are techniques used to improve performance by reducing the initial load time of a React application.

- **Code Splitting:** Breaks the app's bundle into smaller chunks so the browser loads only what's necessary.
- **Lazy Loading:** Loads components only when they are needed, instead of loading everything upfront.

In React, this is often implemented with React.lazy and <Suspense>:

```
const MyComponent = React.lazy(() => import('./MyComponent'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <MyComponent />
    </Suspense>
  );
}
```

36. How would you optimize a slow React application?

Optimizing a React application ensures faster rendering and better performance, especially for large or complex apps:

- Use React.memo, useMemo, and useCallback to avoid unnecessary re-renders.
- Split code with React.lazy and Suspense for on-demand loading.
- Optimize large lists using react-window or react-virtualized.
- Avoid anonymous functions in render; use useCallback instead.
- Keep state localized instead of storing everything globally.
- Use proper key props in lists to prevent unnecessary re-renders.
- For server data, use React Query or SWR with caching to reduce refetching.

37. What is the difference between React.PureComponent and React.Component?

Both `Component` and `PureComponent` are used to create class components, but they handle re-rendering differently:

- **React.Component:** Always re-renders when `setState` is called, regardless of whether the state or props have changed.
- **React.PureComponent:** Implements a shallow comparison of props and state; it only re-renders if something has actually changed.

38. How does React handle re-rendering when state or props change?

Components automatically re-render when their state or props change, but React optimizes this process using the virtual DOM:

- When state or props change, React creates a new Virtual DOM for the component.
- It compares the new Virtual DOM with the previous one (diffing algorithm) to detect changes.
- Only the necessary parts of the real DOM are updated.
- This approach improves performance by avoiding full DOM updates and ensures the UI stays in sync with the data.

39. What is the difference between controlled and uncontrolled components in terms of rendering?

Form inputs can be controlled or uncontrolled, and this affects how they are rendered and updated:

- **Controlled components:** React manages the input's value through state. The UI is always in sync with React state, and updates happen on every change via `onChange`.
- **Uncontrolled components:** The DOM manages the input's value, and React only accesses it when needed using refs. Useful for simple forms where you don't need to track every change.

40. How do useMemo and useCallback help improve performance in React?

In React, re-renders can be expensive if functions or computed values are recreated unnecessarily. `useMemo` and `useCallback` help optimize rendering by caching values and functions:

- **useMemo:** Memoizes the result of a computation so that it is recalculated only when its dependencies change.
- **useCallback:** Memoizes a function so that it is not recreated on every render, preventing unnecessary re-renders in child components.
- Both hooks help avoid expensive recalculations and unnecessary re-renders, improving performance in large or complex components.

React Arrays & Lists Questions:

41. How do you render a list of items in React?

In React, lists are usually rendered using JavaScript's `.map()` method. Each list item should also have a unique `key` prop so React can efficiently track and update elements.

```
const fruits = ["Apple", "Banana", "Mango"];
return (
  <ul>
    {fruits.map((fruit, index) => (
```

```
      <li key={index}>{fruit}</li>
    ))}
  </ul>
);
```

In this Example, .map() is used to iterate over the array, and the key prop helps React identify each list item uniquely.

42. Why is the key prop important in React lists?

The key prop is important because it helps React efficiently manage list rendering. It allows React to identify which items have changed, been added, or removed, instead of re-rendering the entire list.

- Helps React track elements and update only what's necessary.
- Prevents unnecessary re-renders, improving performance.
- Best practice: Use a unique ID as the key instead of the array index whenever possible.

43. What happens if we use the array index as a key in React?

Using the array index as a key technically works, but it is not recommended in most cases. If the list changes (items are reordered, added, or removed), React may reuse components incorrectly, leading to UI bugs.

- Can cause issues when list items are reordered.
- May lead to incorrect updates if items are added or removed.
- Safe only for static lists that never change.

44. How do you conditionally render list items in React?

In React, list items can be conditionally rendered using techniques like if statements, ternary operators, or array methods such as .filter() combined with .map().

Example:

```
const users = [{name: "Aman", active: true}, {name: "Raj", active: false}];
return (
  <ul>
    {users
      .filter(user => user.active)
      .map(user => <li key={user.name}>{user.name}</li>)}
  </ul>
);
```

45. How do you update or remove an item from a list in React state?

State is immutable, so you cannot modify arrays directly. Instead, you create a new array using methods like .map() or .filter() and then update state with that new array.

Example (removing an item):

```
const [list, setList] = useState(["A", "B", "C"]);

const removeItem = (item) => {
  setList(list.filter(i => i !== item));
};
```

In this example, .filter() creates a new array without the removed item, and setList updates the state.

46. What are some performance tips when rendering large lists in React?

Rendering very large lists can hurt performance, so React provides techniques and best practices to optimize them:

- Use unique keys so React can efficiently track items.
- Use virtualization libraries like react-window or react-virtualized to render only the visible portion of the list.
- Memoize list items with React.memo to prevent unnecessary re-renders.
- Implement pagination or lazy loading instead of rendering thousands of items at once.

47. How would you render a nested list in React?

You can nest .map() calls: one for the parent array and another for the child array.

Example:

```
const categories = [
  { name: "Fruits", items: ["Apple", "Banana"] },
  { name: "Veggies", items: ["Carrot", "Tomato"] }
];

return (
  <div>
    {categories.map(cat => (
      <div key={cat.name}>
        <h3>{cat.name}</h3>
        <ul>
          {cat.items.map(item => (
            <li key={item}>{item}</li>
          ))}
        </ul>
      </div>
    ))}
  </div>
);
```

In this example, each level of data gets its own .map() loop, and both parent and child elements should have unique key props.

48. How do you handle dynamic addition of items in a list?

In React, since state is immutable, you cannot directly modify the existing array. Instead, you create a new array that includes the new item and update state.

Example:

```
const [list, setList] = useState(["A", "B"]);

const addItem = (item) => {
  setList([...list, item]);
};
```

In this example:

- [...list, item] creates a new array with the existing items plus the new one.
- Calling setList with this new array updates the state and triggers a re-render, showing the updated list in the UI.

49. How do you handle dynamic sorting or filtering of lists in React?

In React, lists are usually stored in state, and you can sort or filter them dynamically by creating a new array based on the original list and updating the state.

Example:

```
const [users, setUsers] = useState([
  { name: "Aman", age: 25 },
  { name: "Raj", age: 30 },
  { name: "Sara", age: 22 }
]);

// Filter active users
const filteredUsers = users.filter(user => user.age > 24);

// Sort by name
const sortedUsers = [...users].sort((a, b) => a.name.localeCompare(b.name));
```

In this example:

- filter creates a new array containing only the items that meet the condition.
- sort rearranges the array items; using [...users] ensures the original state is not mutated.
- Updating state with the new array triggers a re-render, displaying the sorted or filtered list in the UI.

50. What is the difference between rendering lists with map() vs forEach()?

When rendering lists, it's important to return JSX elements for each item. The choice between map() and forEach() affects this:

- **map():** Returns a **new array**, which can contain JSX elements to be rendered. This is the preferred method for rendering lists in React.
- **forEach():** Does **not return a new array**, so it cannot directly produce elements for rendering. It's only useful for side effects.

Example using map():

```
const fruits = ["Apple", "Banana", "Mango"];
return (
  <ul>
    {fruits.map(fruit => <li key={fruit}>{fruit}</li>)}
  </ul>
);
```

In this example:

- map returns an array of <li> elements, which React renders.
- Using forEach here would not work, because it does not return the array of elements.

React Forms & Events Questions:

51. How are forms handled in React compared to plain HTML?

In plain HTML, the browser's DOM manages form data. In React, form inputs are usually controlled components: in plain html DOM controll and browser will reload the page the input but in react , state controll the input without reloadign the page

- The form values live in state.
- Each keystroke triggers an onChange handler to update state.
- This makes the data predictable and easy to validate before submission.

```
function MyForm() {
  const [name, setName] = React.useState("");
```

```
  const handleSubmit = (e) => {
    e.preventDefault(); //without this browser will reload the page by default and state update
    alert("Submitted: " + name);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <button type="submit">Submit</button>
    </form>
  );
}
```

In this example, the input is fully controlled by React through value and onChange.

52. How do you prevent the default form submission behavior in React?

By using event.preventDefault() inside the form's onSubmit handler.

Example:

```
const handleSubmit = (e) => {
  e.preventDefault();
  console.log("Form submitted!");
};
```

In this example, the page is stopped from reloading and lets React control what happens on submit.

53. What is event bubbling and how can you stop it in React?

- Event Bubbling: When an event triggered on a child element propagates upward to its parent elements.
- In React, you can stop it using event.stopPropagation().

```
const handleClick = (e) => {
  e.stopPropagation();
  console.log("Child clicked, but won't bubble up.");
};
```

54. How do you handle multiple input fields in a single form in React?

By using a single state object and updating it dynamically with name and value.

Example:

```
const [formData, setFormData] = useState({ name: "", email: "" });

const handleChange = (e) => {
  setFormData({ ...formData, [e.target.name]: e.target.value });
};
```

```
import { useState } from "react";

function App() {
 const [name, setName] = useState("");

 return (
  <div>
   <input
    type="text"
    value={name}
    onChange={(e) => setName
(e.target.value)}
   />
   <p>{name}</p>
  </div>
 );
```

This way, one handler can manage multiple inputs.

55. How do you reset form fields after submission in React?

In React, form inputs are usually controlled components, meaning their values are stored in state. To reset a form, you simply reset the state that controls the inputs:

Example:

```
const [formData, setFormData] = useState({ name: "", email: "" });

const handleSubmit = (e) => {
  e.preventDefault();
  console.log(formData);
  // Reset form fields
  setFormData({ name: "", email: "" });
};
```

In this example:

- setFormData({ name: "", email: "" }) resets all input fields to their initial values.
- This triggers a re-render, updating the UI with empty inputs.

React Routing Questions:

56. What is React Router and why is it used?

React Router is a library that enables client-side routing in React applications. It allows you to create single-page applications (SPAs) where navigation between views happens without a full page reload, making apps faster and smoother.

57. Difference between <a> tag and <Link> in React Router?

- <a> tag: Reloads the whole page when navigating.
- <Link> component: Updates the URL and changes the component without reloading (SPA behavior).

58. What are the main components of React Router?

- <BrowserRouter> – Wraps the app and enables routing.
- <Routes> – Container for all route definitions.
- <Route> – Defines a path and the component to render.
- <Link> / <NavLink> – For navigation.
- useNavigate() – For programmatic navigation.

59. What is the difference between useHistory and useNavigate?

- In React Router v5 : navigation was done using useHistory().
- In React Router v6 : replaced with useNavigate().

```
const navigate = useNavigate();
navigate("/dashboard");
```

60. How do you implement nested routes in React Router?

```
<Routes>
  <Route path="/dashboard" element={<Dashboard />}>
    <Route path="profile" element={<Profile />} />
    <Route path="settings" element={<Settings />} />
  </Route>
</Routes>
```

/dashboard/profile : shows Profile inside Dashboard.

61. What are route parameters and how do you access them?

Route params are dynamic parts of the URL.

```
<Route path="/users/:id" element={<User />} />
```

Inside User component:

```
import { useParams } from "react-router-dom";
const { id } = useParams();
```

/users/101 : id = 101

62. How do you redirect a user in React Router?

- Using <Navigate> component:

```
<Route path="/old" element={<Navigate to="/new" />} />
```

Using useNavigate() hook for programmatic redirects.

63. What is the difference between client-side routing and server-side routing?

Client-Side Routing

- In client-side routing, only the first request loads the HTML + JS bundle.
- After that, navigation happens inside the browser using JavaScript (like React Router).
- The page doesn't reload : only the required component/view updates.

Example Flow:

- User visits /about.
- Server sends index.html + JS bundle (React app).
- React Router detects /about and loads <About /> component without reloading the page.

Server-Side Routing

In server-side routing, whenever a user clicks a link or enters a URL:

- A request goes to the server.
- The server processes it and sends back a new HTML page.
- The browser reloads the page completely.

Example Flow:

- User visits /about.
- Browser sends a request to server : /about.
- Server responds with an about.html page.
- Browser reloads and shows About page.

64. Difference between <Switch> and <Routes> in React Router

Switch:

- Used in React Router v5 to render only the first matching route.
- Checks routes from top to bottom and renders the first match.
- If multiple routes match, only the first one is rendered.

- The order of routes is very important.
- Supported component, render, and children props.
- Needed the exact keyword to avoid unwanted matches.
- Nested routes were harder to manage.

Example:

```
<Switch>
  <Route exact path="/" component={Home} />
  <Route path="/about" component={About} />
  <Route path="/contact" render={() => <Contact />} />
</Switch>
```

Routes:

- Introduced in React Router v6 as a replacement for <Switch>.
- Uses a ranking algorithm to automatically find the best match.
- The order of routes does not matter.
- Supports only the element prop for rendering.
- Cleaner and simpler syntax compared to <Switch>.
- No need for the exact keyword anymore.
- Makes nested routing much easier.
- <Switch> is deprecated, and <Routes> is now the recommended way.

Example:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
</Routes>
```

This React Interview Questions and Answers covers a wide range of topics, from basic concepts to advanced techniques. Whether you're a beginner or an experienced developer, mastering these questions will enhance your readiness for React interviews and boost your confidence.

> *For further reading, check out our dedicated article on **Advanced ReactJS Intermediate Interview Questions**.*

Comment S shobhi... + Follow 67

Article Tags: Web Technologies ReactJS Interview Questions React-Questions +1 More

GeeksforGeeks Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Careers
Contact Us
Corporate Solution

Explore
POTD
Practice Problems
Connect
Blogs
90% Refund on Courses

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects

Courses
ML and Data Science
DSA and Placements
Web Development
Data Science
Programming Languages

Offline Centers
Noida
Bengaluru
Pune
Hyderabad
Kolkata

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design



Training Program

GATE School Subjects Software and Tools

DevOps & Cloud GATE Trending Technologies

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved